

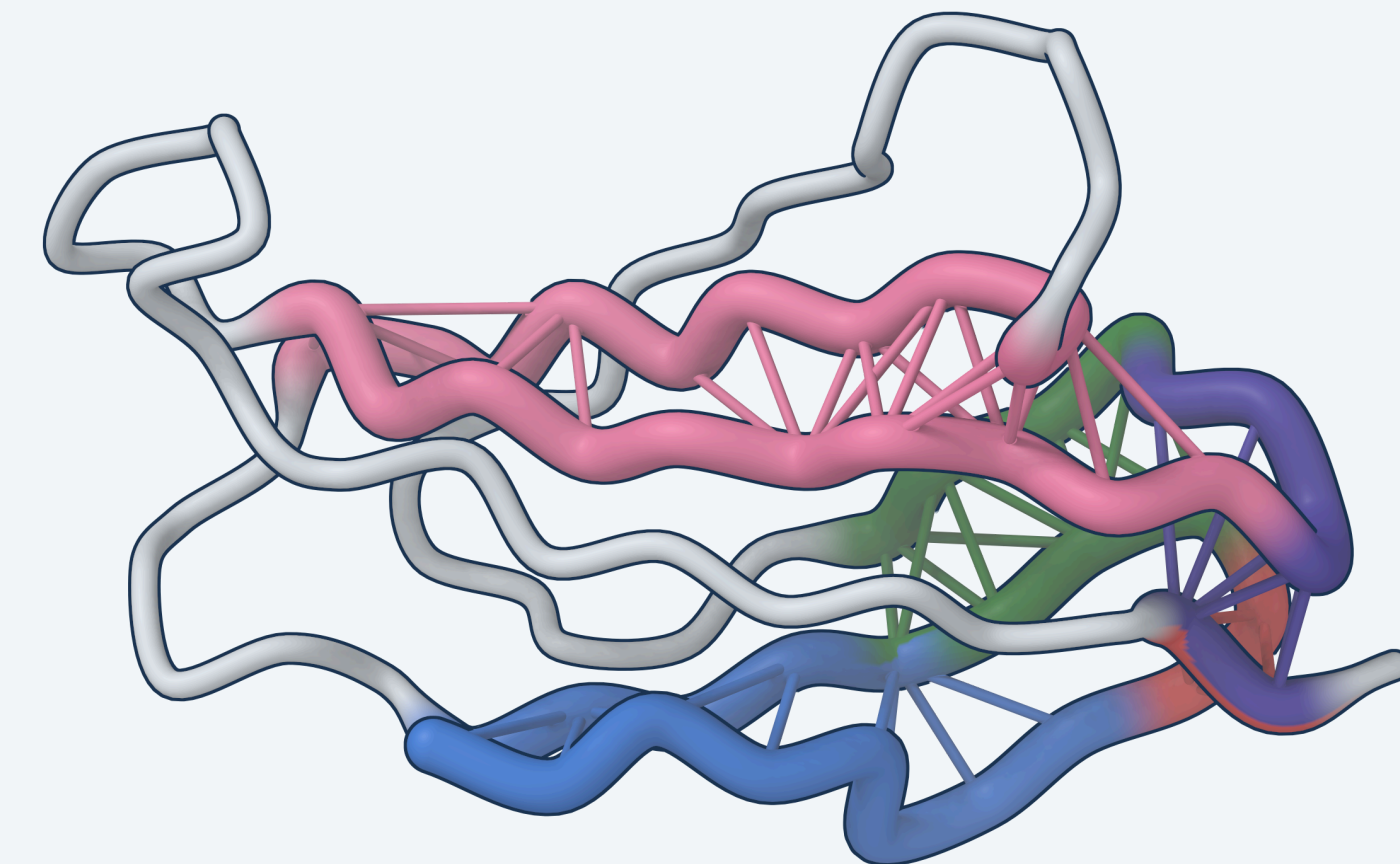
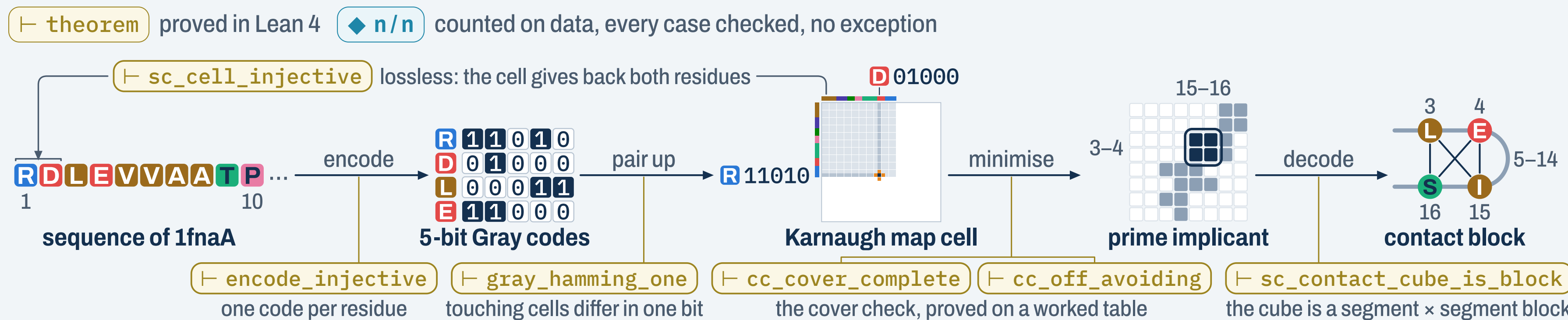
# Is KMAP Possible? Formal Verification of Karnaugh Map Encoding for Biological Sequences in Lean 4

Shuvam Banerji Seal, Susmita Roy\*, Dwaipayan Roy  
Indian Institute of Science Education and Research Kolkata, India  
\*susmita.roy@iiserkol.ac.in

## 000 The finding

We formalised in Lean 4 the mathematics that turns DNA, RNA and protein sequences into Karnaugh maps: 131 machine-checked theorems, none left unproved. On this foundation every structure becomes an exact Boolean circuit whose rules hold without exception, for proteins, a viral species and folding simulations.

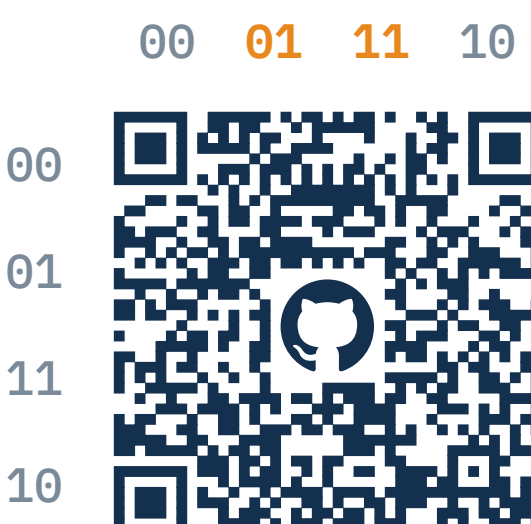
Gray-coded grids of sequences, from Petoukhov's genetic Karnaugh matrices<sup>1</sup> to chaos-game maps (FCGR)<sup>2,3</sup>, are usually taken on trust. Here the encodings are theorems and every circuit and rule is checked on every case.



Strand pairs, found as blocks

- 25-34 with 61-70: 6 blocks
- 1-8 with 14-18: 4 blocks
- 11-14 with 53-56: 3 blocks
- 57-60 with 89-90: 2 blocks
- 9-10 with 89-90: 1 block

### Turn any protein into a circuit



Any PDB entry, step by step, to its rules and inferences.

## 001 A code that changes one bit at a time

Binary counting can flip many bits at once: 0111 → 1000 flips four. The reflected Gray code  $g(n) = n \oplus (n \gg 1)$  visits every number once and changes exactly one bit per step. DNA needs two bits, and A C G T is itself a Gray cycle.

| n | binary | flips | Gray $g(n)$ | flips | DNA needs two bits |
|---|--------|-------|-------------|-------|--------------------|
| 0 | 000    |       | 000         |       | 00 A 01            |
| 1 | 001    | 1     | 001         | 1     | 00 A 01            |
| 2 | 010    | 2     | 011         | 1     | 00 A 01            |
| 3 | 011    | 1     | 010         | 1     | 00 A 01            |
| 4 | 100    | 3     | 110         | 1     | 10 T 11            |
| 5 | 101    | 1     | 111         | 1     | 10 T 11            |
| 6 | 110    | 2     | 101         | 1     | 10 T 11            |
| 7 | 111    | 1     | 100         | 1     | 10 T 11            |

gray\_hamming\_one gray\_injective gray\_cycle\_AC...TA

## 011 A Karnaugh map<sup>4</sup> is a folded truth table

Order a grid's rows and columns by the Gray code and touching cells differ in one variable, even across the edges, which wrap like a torus. Put a  $k$ -mer's first letters on the rows and the rest on the columns: each 2-mer owns one cell of a 4×4 map, each 4-mer one of a 16×16 map.

| first base | second base | A 00 | C 01 | G 11 | T 10 |
|------------|-------------|------|------|------|------|
| A 00       |             | AA   | AC   | AG   | AT   |
| C 01       |             | CA   | CC   | CG   | CT   |
| G 11       |             | GA   | GC   | GG   | GT   |
| T 10       |             | TA   | TC   | TG   | TT   |

One cell per  $k$ -mer  
4 × the cells per base

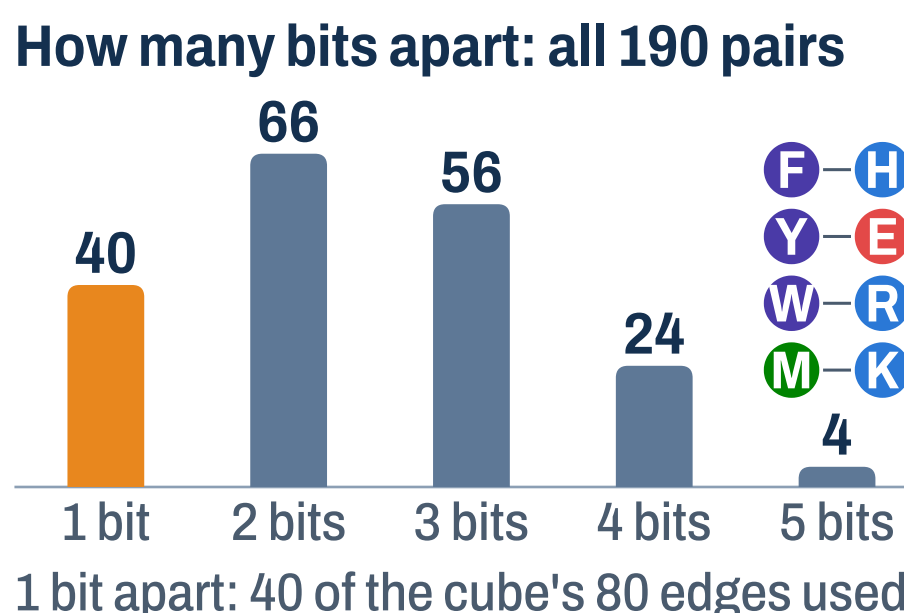
$k = 1$   $k = 2$   $k = 3$   $k = 4$   
2 × 2 4 × 4 8 × 8 16 × 16

dinucEncode\_injective tetranucEncode\_injective gray\_code\_preserves\_adjacency

## 010 Amino acids on a five-dimensional cube

Proteins need five bits: 32 codewords for 20 residues. Numbering residues group by group before Gray-coding keeps chemistry close. All 190 pairwise distances are proved: 40 pairs sit one bit apart, only four sit five apart, and the code uses exactly half of the cube's 80 edges.

| first 2 bits | 000 | 001 | 010 | 011 | 100 | 101 | 110 | 111 | 101 | 100 |
|--------------|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|
| 00           | A   | V   | L   | I   | F   | Y   | W   | M   |     |     |
| 01           | D   | Q   | N   | T   | S   | G   | P   | C   |     |     |
| 11           | E   | H   | K   | R   |     |     |     |     |     |     |
| 10           |     |     |     |     |     |     |     |     |     |     |



encode\_injective unorderedDistanceDistribution max\_distance\_FH...MK  
encoding\_edge\_coverage

## 110 Contacts as a Boolean function

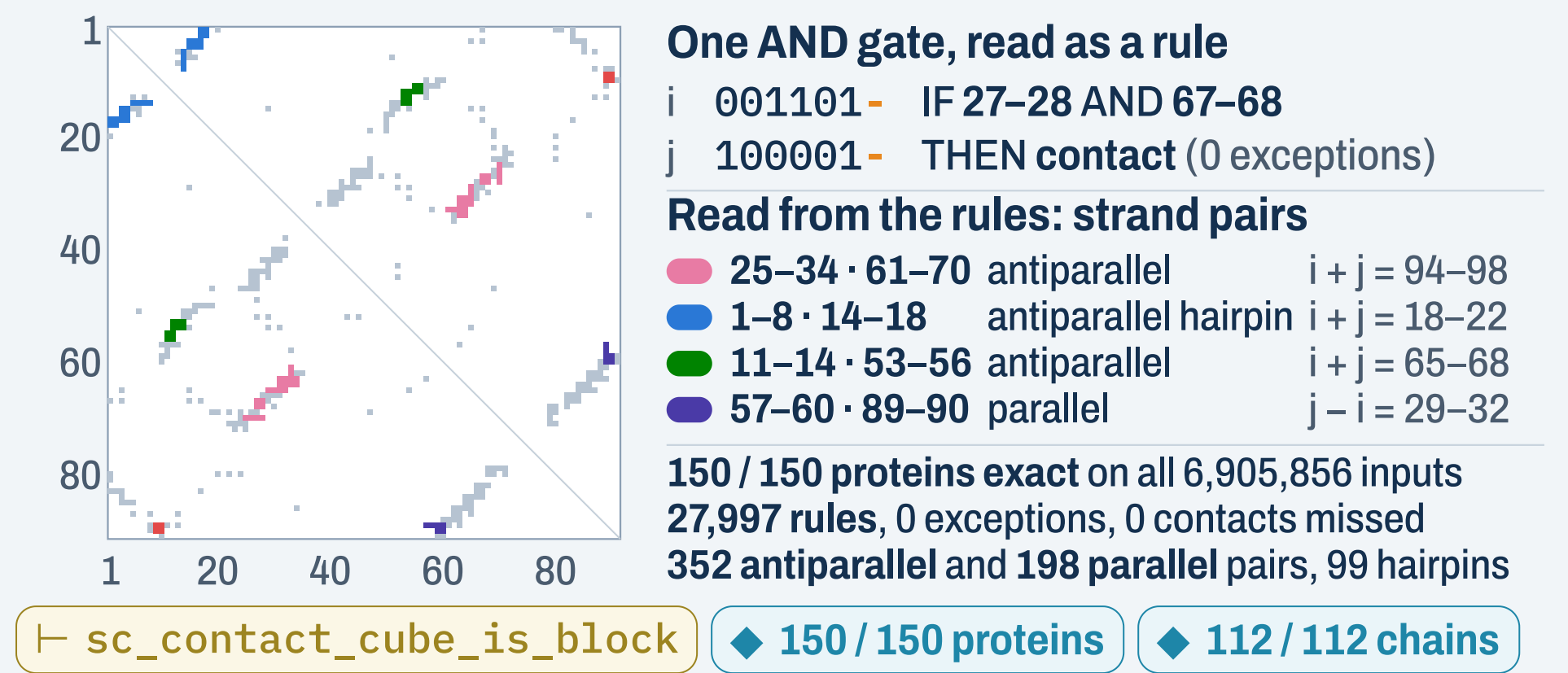
A contact map is a Boolean function:  $C(i, j) = 1$  when residues  $i$  and  $j$  lie within 8 Å. Quine-McCluskey<sup>5</sup> merges 1-cells one bit apart into ever larger cubes; the cubes that cannot grow are the prime implicants, and a cover of them is a two-level circuit equal to the map. For an 8-residue example Lean proves the map symmetric and that no two of its contacts can merge.

| Merge cells one bit apart                                                  | 8 residues, 4 contacts                                                                                                                                                        |
|----------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| AB 00 01 11 10<br>00 01 11 10<br>01 11 11 10<br>11 11 11 10<br>10 11 11 10 | 0101+0111 → 01-1<br>1101+1111 → 11-1<br>01-1+11-1 → -1-1<br>1000+1001 → 100-<br>f = BD + ABC<br>dashed: prime, not needed<br>no two touch, so each is its own prime implicant |

contactMap8\_symmetric contactMap\_no\_groupable\_minterms cc\_cover\_complete

## 100 Rules with no exception, from exact circuits

Each PSICOV<sup>7</sup> protein becomes a circuit equal to its contact map. Its AND gates are rules; block rules that continue one another give strand pairs, direction and register, by counting.



One species: in 1,299 SARS-CoV-2 Spike sequences, exact circuits list which residue pairs occur at co-evolving positions and which never do. No sequence breaks a rule.

| positions 371 × 376<br>1,294 sequences                                                                                                                    | positions 405 × 408<br>1,289 sequences                 | positions 486 × 493<br>1,297 sequences                     |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------|------------------------------------------------------------|
| A376 T376<br>F371 970 0<br>L371 0 311<br>S371 1 12                                                                                                        | R408 S408<br>D405 319 0<br>N405 12 958                 | Q493 R493<br>F486 20 1,173<br>P486 2 0<br>V486 102 0       |
| F371 ⇒ A376 in 970 of 970<br>L371 ⇒ T376 in 311 of 311                                                                                                    | D405 ⇒ R408 in 319 of 319<br>S408 ⇒ N405 in 958 of 958 | V486 ⇒ Q493 in 102 of 102<br>R493 ⇒ F486 in 1,173 of 1,173 |
| 10 position pairs · 41 allowed and 35 forbidden residue combinations · 23 implications<br>0 violations: every one of the 1,299 sequences obeys every rule |                                                        |                                                            |

1,299 / 1,299 sequences obey every rule unique minimum covers

### Takeaways

- A verified foundation. Lean fixes the encodings and what every cube means; every circuit is exact on all inputs.
- Rules that are true, not likely. Each holds for every contact, sequence or frame it names.
- One language for sequence, structure and dynamics. Any finite encoding can be checked the same way.

Try any PDB entry, from file to circuit, rules and inferences:  
e-motioner-x-sbs.github.io/ICMS\_Submission

## 111 How Lean checks a claim

In Lean 4<sup>6</sup> a statement is a type and a proof is a program checked by a small kernel. For finite claims, native\_decide evaluates every case in compiled code, up to 2<sup>20</sup> in one theorem, and so also trusts Lean's compiler. No proof uses sorry or adds an axiom of its own.

```
-- Gray code is additive over GF(2)
theorem sc_gray_gf2_linear :
  ∀ x y : Fin 32,
    scGray (x.val ^^^ y.val)
      = scGray x.val ^^^ scGray y.val
:= by native_decide
```

axiom audit: 131 theorems, 0 sorry Python ↔ Lean bridge: 156/156 checks

| native_decide          | standard axioms only | no axiom     |
|------------------------|----------------------|--------------|
| KmapProofs             | 16                   |              |
| KmerIndexing           | 35                   |              |
| AminoAcidEncoding      | 27                   |              |
| ContactMapCompleteness | 20                   |              |
| KmapEncodingEquiv      | 8                    |              |
| ContactCircuits        | 12                   | 131 theorems |
| SequenceCircuits       | 13                   | 100 + 23 + 8 |

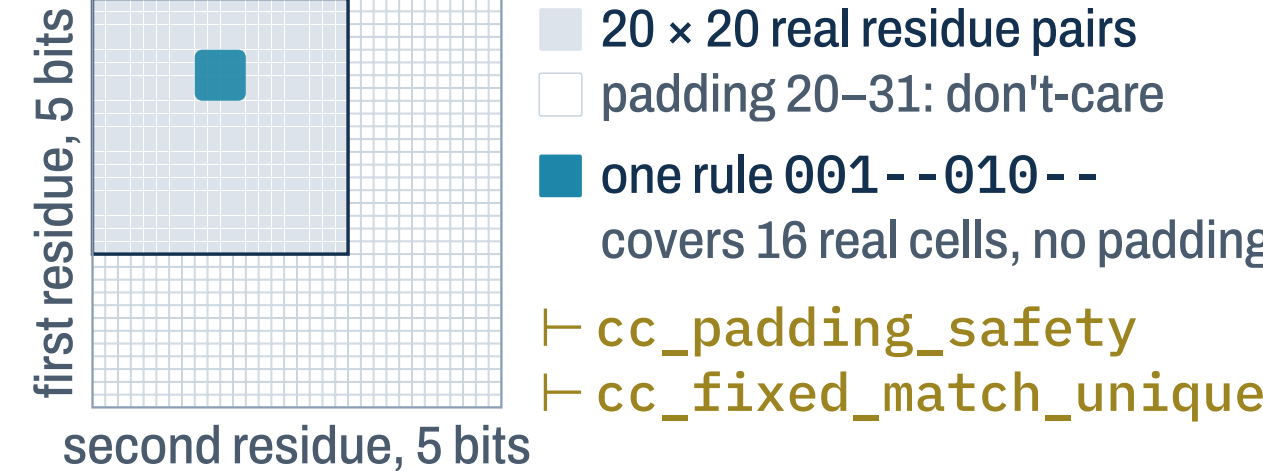
## 101 What the proofs guarantee

### Exact rules on padded maps

Twenty residues need five bits per axis, so a 20×20 residue-pair map is padded to 32×32 with 10-bit addresses. Lean proves that a fully fixed rule matches exactly its own cell and that any rule on real residues fires only inside the 20×20 block: padding never enters a result.

cc\_padding\_safety cc\_fixed\_match\_unique cc\_off\_avoiding

32 × 32 = 1,024 addresses (10 bits)



### Segments hold under either numbering

With its low position bits free, a cube is an unbroken chain segment. Lean proves this for Gray-coded positions as well as plain binary: the two numberings name different segments, never broken ones.

sc\_gray\_cube\_is\_also\_interval sc\_gray\_relabels\_segments

Same cube, still a segment, different residues

| Plain binary                                    | cube 10 - - → residues 8-11  |
|-------------------------------------------------|------------------------------|
| 00 00 00 00 01 01 01 01 10 10 10 10 11 11 11 11 |                              |
| 00 01 10 11 00 01 10 11 00 01 10 11 00 01 10 11 |                              |
| 01-1-2-3-4-5-6-7-8-9-10-11-12-13-14-15          |                              |
| 00 00 00 00 01 01 01 01 11 11 11 11 10 10 10 10 |                              |
| 00 01 11 10 10 11 01 00 00 01 11 10 10 11 01 00 |                              |
| Gray-coded                                      | cube 10 - - → residues 12-15 |

### What the encoding can and cannot change

Since  $g(x \oplus y) = g(x) \oplus g(y)$ , a Gray-ordered map is a lexicographic (FCGR) map with its cells permuted: every statistic of the cell values is identical. The code changes only distances between letters: of the 400 ordered amino-acid pairs, exactly 80 are one bit apart.

sc\_gray\_gf2\_linear cc\_he\_dist\_counts 120 / 120 maps: same cell values

| Lexicographic (FCGR)      | Gray      | Same 16 counts, moved |
|---------------------------|-----------|-----------------------|
| A C G T i → g(i)          | A C T G   | CGCGAATTCGCG          |
| A 1 1 1 1 00 → 00 1 1 1 1 | A 1 1 1 1 | (1BNA), address order |
| C 1 2 4 10 → 01 10        | C 1 2 4   |                       |
| G 1 2 4 10 → 10 10        | G 1 2 4   |                       |
| T 1 1 1 1 11 → 11 11      | T 1 1 1 1 |                       |

What Gray changes: code distance d, 400 residue pairs

| d = 0 | d = 1 | d = 2 | d = 3 | d = 4 | d = 5 | 80 / 400 pairs one bit apart |
|-------|-------|-------|-------|-------|-------|------------------------------|
| 20    | 80    | 132   | 112   | 48    | 8     |                              |

### Rules that hold in every frame of a folding run

In structure-based-model<sup>8</sup> trajectories every native contact is formed or broken in every frame. Rules “whenever  $a$  is formed, so is  $t$ ” that no frame breaks join neighbouring contacts; in ten long 8,000-frame runs, all of them do.

9,986 rules, 0 counterexample frames 479 trajectories, 48,385 frames

| Crambin, 8,000 frames      | 9,986 rules hold in every frame      |
|----------------------------|--------------------------------------|
| a = contact 4-44, t = 4-43 | of 479 folding runs                  |
| t formed t broken          | like a → t: a formed never without t |
| a formed 5,429 0           |                                      |
| a broken 1,258 1,313       |                                      |

3,966 join neighbouring contacts 33 / 33 in the long runs

1. S. V. Petoukhov, *BioSystems* **2024**, 246, 105349.  
2. H. J. Jeffrey, *Nucleic Acids Res.* **1990**, 18, 2163-2170.  
3. J. S. Almeida, S. Vinga, *BMC Bioinformatics* **2009**, 10, 100.

4. M. Karnaugh, *Trans. AIEE* **1953**, 72, 593-599.  
5. W. V. Quine, *Am. Math. Mon.* **1952**, 59, 521-531; E. J. McCluskey, *Bell Syst. Tech. J.* **1956**, 35, 1417-1444.

6. L. de Moura, S. Ullrich, *CADE-28*, LNCS 12699, **2021**, 625-635.  
7. D. T. Jones et al., *Bioinformatics* **2012**, 28, 184-190.  
8. J. K. Noel et al., *PLoS Comput. Biol.* **2016**, 12, e1004794.